

Name: Jaykishan J Saharan

Div: D Roll no.: 26

Course/Program: BTech CSE SY

Subject: DSA

DSA lab assignment 10

* Title: To implement Depth First Search / Breadth First Search Algorithm

Q.1) What are the key differences between DFS and BFS in terms of their traversal order and data structures used?

Solⁿ: Traversal order:

- DFS (Depth-First Search): Explores the graph by going as deep as possible along each branch before backtracking. It starts at a source node, then explores one of its unvisited neighbours, then that neighbor's unvisited neighbor, and continues this path until it reaches a node with no visited neighbors. Only then does it backtrack to explore other branches.

BFS (Breadth-First Search): Explores the graph level by level. It starts at a source node, then visits all its immediate neighbors, then all their unvisited neighbors, and so on. This means it prioritizes exploring nodes closer to the source before moving to deeper levels.

Data Structures used:

- DFS: Employs a stack data structure, either explicitly or implicitly through recursion. Nodes are pushed onto the stack when discovered and popped off to be processed.

This "Last-In, first-out" (LIFO) behavior allows for deep exploration along a single path before backtracking.

- 1) BFS: Utilizes a queue data structure. Nodes are added to the back of the queue when discovered and removed from the front to be processed. This "First-In, First-Out" (FIFO) behavior naturally facilitates level-by-level traversal.

Q.2) How does the time complexity of DFS and BFS change with different graph representations (adjacency list vs. adjacency matrix)?

Solⁿ:-

1. Adjacency List representations.

- Time complexity: $O(V + E)$, where V is the number of vertices and E is the number of edges.
- Reasoning: When using an adjacency list, finding all neighbors of a vertex involves iterating through its corresponding list. The sum of the lengths of all adjacency lists in an undirected graph is $2E$ (Each edge appears twice), and E in a directed graph. Since each vertex is visited once and each edge is traversed a constant number of times (once for directed, twice for undirected), the total time complexity is proportional to the sum of vertices and edges.

2. Adjacency matrix Representations:

- Time complexity: $O(V^2)$, where V is the no. of vertices.

- Reasoning: When using an adjacency matrix, to find all neighbors of a vertex, one must iterate through an entire row (or column) of the $V \times V$ matrix. This operation takes $O(V)$ time for each vertex. Since each of the V vertices is visited, the total time complexity becomes $O(V \times V) = O(V^2)$.

Q.3) What are the practical applications of DFS and BFS in real-world applications problems?

Sol:

Breadth-First Search (BFS) applications:

- Shortest Path: finds the shortest path in unweighted graphs, useful for social networks (e.g., friends or friends of friends) and GPS navigation systems to find the nearest neighbors.
- Web crawling: used for search engines to index the web by exploring pages level by level from a starting point, ensuring all pages at a certain depth are visited before moving deeper.
- Network Broadcasting: Efficient for sending a message to all nodes in a network starting from a single source node.
- Garbage collection: used in algorithms like Cheney's algorithm for copying garbage collection due to its better locality of reference.
- Cycle Detection: can detect cycles in undirected graphs.
- Maximum Flow: BFS is preferred in the Ford-Fulkerson algorithm for finding maximum flow, as it can reduce the worst-case time complexity.

Depth-First Search (DFS) applications.

- Maze Solving: Explores a path as deeply as possible before backtracking, making it suitable for finding a path through a maze.
- Cycle Detection: Used to detect cycles in directed and undirectional graphs.
- Topological Sorting: orders tasks with dependencies, which is crucial for scheduling.
- Finding connected components: Identifies all connected components in a graph.
- Pathfinding: can be used to find any path between two vertices.
- Puzzles: Excellent for solving constraint satisfaction problems, such as crosswords and Sudoku.

Q.4) How would you modify the DFS algorithm to detect cycles in a graph?

Solⁿ: To modify the Depth-First Search (DFS) algorithm for cycle detection in a graph, additional state tracking for nodes is required. The specific modification depends on whether the graph is directed or undirected.

for directed Graph.

- Maintain two arrays/sets:
- visited: Tracks all nodes that have been visited at any point during the DFS traversal.
- recursionStack: Tracks nodes currently in the recursion stack of the current DFS path.

• DFS Traversal:

- when a node u is visited, mark it as visited and add it to recursionstack.
- for each neighbor v of u :
 - if v is not visited, recursively call DFS on v . If this recursive call returns true (indicating a cycle), propagate true upwards.
 - if v is visited AND v is also in recursionstack, then a back edge is found, indicating a cycle. Return true.
- After exploring all neighbors of u , remove u from recursionstack before returning false (if no cycle was found in this path).

For Undirected Graph.

- Maintain a visited array/set and pass the parent node in the DFS function.
- DFS Traversal:
 - when a node u is visited, mark it as visited.
 - for each neighbor v of u :
 - if v is not visited, recursively call DFS on v , passing u as the parent. If this recursive call returns true, propagate true upwards.
 - if v is visited AND v is not the parent of u , then a back edge (other than the one connecting u to its immediate parent) is found, indicating a cycle. Return true.
 - If no cycle is found after exploring all neighbors, return false.

Q.5) can DFS or BFS be used to find the shortest path in an unweighted graph? if yes, which one and why?

Sol:-

Yes, BFS can be used to find the shortest path in an unweighted graph; while DFS cannot guarantee the shortest path.

Breadth-first search (BFS):

How it works: BFS starts at the source node and explores all its neighbors (at distance 1), then all the neighbors of those neighbors (at distance 2), and so on.

Why it finds the shortest path:

By exploring nodes layer by layer, the first time BFS reaches the destination node, it is guaranteed to have found the shortest possible path, as it has already explored all nodes at shorter distances.